Software Requirements Specification (SRS)

The Queue

Revised Final Submission

Prepared by: Daysha Roberts

Course: COSC 4308

Repository: https://github.com/xTop5ive/The-Queue

Date: May 5, 2026

1. Introduction

The Queue is a playlist-sharing and collaborative social media web application focused on music identity, playlist discovery, and community engagement. The project began as a music-centered idea and developed into a platform where users can create, browse, and interact with playlists in a way that feels more social than a traditional music streaming service.

This revised SRS documents the current project scope, requirements, architecture, user expectations, development progress, and future improvements for the final checkpoint submission. It reflects the current implementation and the planned direction of the project.

1.1 Purpose

The purpose of this document is to define the requirements and current design of The Queue. It serves as a reference for the final project artifacts, including source code, user documentation, setup instructions, deployment information, and presentation materials.

1.2 Scope

The Queue allows users to interact with playlists as social objects. Users can view public playlists, create playlist entries, add tags, upload cover art, browse content, search for playlists, and interact with playlists through likes or detail pages. The long-term vision includes deeper community features, user-created communities, onboarding, profile matching, and collaborative music discovery.

1.3 Intended Audience

- Course instructor and project evaluators reviewing the final checkpoint.
- Developers who may continue building or maintaining the project.
- Students learning how the system was designed and implemented.
- Potential users who want to understand the app purpose and features.

2. Overall Description

2.1 Product Perspective

The Queue is a standalone web application built with modern web development tools. It uses a client-facing interface for users and backend services for authentication, database storage, and uploaded media. The system is designed to be expanded over time as more playlist, profile, and community features are added.

2.2 Product Functions

- Allow users to sign up, sign in, and sign out.
- Allow users to create and view playlists.
- Allow users to add playlist titles, descriptions, tags, and cover images.

- Allow users to browse and search public playlists.
- Allow users to open playlist detail pages.
- Allow users to like playlists.
- Support a profile page foundation that can be connected to real user data.
- Support future music-based communities and sub-communities.
- Support future onboarding questions to help place users in the right music spaces.

2.3 User Classes

User Class	Description	Main Needs
General User	A person who uses the app to discover and share playlists.	Browse playlists, like playlists, create profile, discover new music.
Playlist Creator	A user who builds and shares curated playlists.	Create playlist entries, add tags, upload cover art, share links.
DJ or Curator	A music-focused user who organizes music for events, moods, or audiences.	Organize playlists by vibe, genre, audience, and setting.
Admin or Developer	A person maintaining the application.	Review code, manage setup, troubleshoot database and deployment issues.

3. Functional Requirements

ID	Requirement	Priority	Status
FR-1	The system shall allow users to create an account and authenticate through the application.	High	Implemented or partially implemented through Supabase Auth.
FR-2	The system shall allow users to create playlists with a title, description, tags, and cover image.	High	Implemented or in current build.
FR-3	The system shall allow users to browse public playlists.	High	Implemented or in current build.
FR-4	The system shall allow users to search or filter playlists.	Medium	Implemented or partially implemented.
FR-5	The system shall allow users to open a playlist detail page.	High	Implemented or partially implemented.
FR-6	The system shall allow users to like playlists.	Medium	Implemented or partially implemented with database connection.
FR-7	The system shall support profile pages for user identity.	High	In progress.
FR-8	The system shall support music communities and sub-communities.	Medium	Planned or partially designed.
FR-9	The system shall support onboarding questions for community matching.	Medium	Planned.

FR-10	The system shall provide setup documentation for developers.	High	Included in final artifacts.

4. Non-Functional Requirements

ID	Requirement	Explanation
NFR-1	Usability	The app should be easy to navigate with clear pages, buttons, forms, and playlist cards.
NFR-2	Performance	Pages should load within a reasonable time and avoid unnecessary delays during browsing.
NFR-3	Security	Authentication should be handled through Supabase and secret keys should not be committed to GitHub.
NFR-4	Maintainability	Code should be organized into clear folders, reusable components, and readable files.
NFR-5	Scalability	The design should allow future expansion into communities, onboarding, and social features.
NFR-6	Portability	The project should be able to run locally using documented installation instructions.

5. System Architecture

The Queue uses a modern web application architecture. The frontend is built with Next.js and React components. Styling is handled with Tailwind CSS. Supabase supports authentication, database operations, and storage. Prisma or database configuration files help define and connect application data where applicable.

Layer	Responsibility	Technology
Client Layer	Displays pages, forms, playlists, navigation, and user interface components.	Next.js, React, Tailwind CSS
Application Layer	Handles routing, page logic, server actions or API routes, and playlist interactions.	Next.js App Router
Data Layer	Stores users, playlists, likes, tags, and related records.	Supabase, PostgreSQL
Storage Layer	Stores uploaded media such as playlist cover images.	Supabase Storage
Version Control	Tracks code changes and project history.	GitHub

6. Database Design

The database is designed to support users, playlists, tags, likes, and future community features. The exact structure may continue to evolve as development continues, but the system is centered on connecting users to playlists and playlist interactions.

Entity	Purpose	Example Fields
User/Profile	Stores user identity and profile details.	id, username, display_name, profile_photo, bio
Playlist	Stores playlist information created by users.	id, user_id, title, description, cover_url, created_at
Tag	Stores playlist categories or vibe labels.	id, name
Playlist_Tag	Connects playlists to tags.	playlist_id, tag_id
Like	Tracks playlist likes by users.	id, user_id, playlist_id, created_at
Community	Future table for music communities.	id, name, description, category

7. User Interface Requirements

- The interface should present a clear home, feed, explore, playlist, and profile experience.
- Playlist cards should show title, cover image, tags, and basic interaction options.
- Forms should provide clear labels and feedback when users create or update content.
- The design should support a modern music platform feel with a polished visual identity.
- Screens should remain readable on common laptop and desktop display sizes.

8. Constraints

- The project must be completed within the course timeline.
- The final submission must include a GitHub repository, documentation, source code, setup files, user guide, and final presentation.
- Authentication and database behavior depend on properly configured Supabase environment variables.
- Some planned social features may remain incomplete due to time limits.
- The project must avoid exposing private keys or database secrets in GitHub.

9. Assumptions and Dependencies

- Users will have internet access to use the deployed version or access the GitHub repository.
- Developers will install Node.js and npm before running the project locally.
- Supabase must remain available for authentication, database, and storage features.
- The GitHub repository will remain accessible to the instructor through collaborator access.
- The app can continue to grow after submission through future commits and feature updates.

10. Planned vs. Actual Progress

The project timeline changed as the idea became more focused. Early planning involved shaping the concept and deciding how music, playlists, and social features should work together. Development then moved into building the Next.js application, setting up Supabase, creating playlist features, and preparing final documentation.

Phase	Planned Work	Actual Progress
Planning	Define problem, users, scope, and app direction.	Completed. The app direction was refined into a playlist-sharing social platform.
Design	Plan UI, playlist flow, profile flow, and community concepts.	Completed with some revisions as the app evolved.
Development	Build authentication, playlist creation, playlist browsing, and interaction features.	Partially completed. Core playlist and app structure were built, while some profile and community features remain in progress.
Testing	Test key pages, forms, and database behavior.	Partially completed through development testing and troubleshooting.
Documentation	Prepare SRS, install guide, user guide, troubleshooting, FAQ, and final presentation.	Completed for final checkpoint submission.

11. Current Implementation Status

The current version of The Queue demonstrates the main foundation of a playlist-sharing application. It includes the application structure, styling, playlist browsing direction, playlist creation direction, authentication setup, and database-backed interactions. The app also includes planning for profile pages, communities, onboarding, and expanded social features.

11.1 Completed or Working Areas

- Next.js project structure and routing foundation.
- Tailwind CSS styling and visual layout direction.
- Supabase integration for backend services.
- Playlist browsing and playlist detail concepts.
- Playlist creation with music-focused information such as tags and cover art.
- Like button or playlist interaction foundation.
- GitHub repository with progressive development commits.

11.2 In Progress Areas

- Profile page connection to real user data.
- Expanded community and sub-community features.
- More complete onboarding process for new users.
- Improved feed purpose and social interaction design.
- More complete testing and deployment polish.

11.3 Not Started or Future Features

- User-created communities.
- Advanced matching based on music taste, rhythm patterns, or listening habits.
- Collaborative playlist editing.
- Notifications and messaging.
- Admin moderation tools.
- Mobile app version.

12. Risk Management

Risk	Impact	Mitigation
Time limitations	Some planned features may remain incomplete.	Prioritize core playlist and documentation requirements.
Database setup issues	App may not run without correct environment variables.	Provide clear setup files and avoid committing secrets.
Scope creep	Too many social features could make the project too large.	Focus final submission on what exists and list future features separately.
Deployment problems	Live demo may not work if environment variables are missing.	Provide local installation instructions as backup.

13. Future Improvements

- Complete the multi-step onboarding flow.
- Allow users to create their own music communities.
- Build stronger profile customization, including profile photos and music identity questions.
- Improve the feed so it shows useful playlist activity and community updates.
- Add playlist collaboration between users.
- Improve recommendation logic using tags, genres, moods, and eventually rhythm-based matching.
- Add more polished mobile responsiveness and accessibility support.

14. Conclusion

The Queue is a strong foundation for a music-centered social platform. It combines playlist sharing, user identity, discovery, and future community interaction into one focused application. The current implementation demonstrates the major direction of the project while leaving room for continued development after the course. The final artifacts provide the instructor with the source code, documentation, setup instructions, user guidance, and presentation materials needed to evaluate the project.

Appendix A: Final Artifact Checklist

Artifact	Repository Location
Revised SRS PDF	documentation/The_Queue_SRS_Revised.pdf
Revised SRS Source File	documentation/The_Queue_SRS_Revised.docx
Planned vs. Actual Gantt Chart	documentation/Gantt_Chart_Planned_vs_Actual.pdf
Source Code	src/ or source-code/
Installation Instructions	deployment/INSTALL.md
User Guide	user-docs/USER_GUIDE.md
Troubleshooting and FAQ	user-docs/
Final Presentation	presentation/The_Queue_Final_Presentation.pptx or Canvas upload